Published: 25 November 2025

RESEARCH-ARTICLE

End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks

SUNGHYUN JIN, Seoul National University, Seoul, South Korea

SERAE KIM, Seoul National University, Seoul, South Korea

SANGTAE HA, University of Colorado Boulder, Boulder, CO, United States

KYUNGHAN LEE, Seoul National University, Seoul, South Korea

Proceedings of the ACM on Networking, Volume 3, Issue CoNEXT4 (November 2025)
https://doi.org/10.1145/3768987
EISSN: 2834-5509

End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks

SUNGHYUN JIN, Seoul National University, Republic of Korea
SERAE KIM, Seoul National University, Republic of Korea
SANGTAE HA, University of Colorado Boulder, USA
KYUNGHAN LEE, Seoul National University, Republic of Korea

The growing adoption of deep neural network (DNN) inference in mobile applications underscores the need for edge-assisted inference to meet the latency requirements of diverse DNN tasks, given the constrained device capabilities. However, widespread deployment remains hindered by unreliable latency due to the shared nature of cellular networks, where concurrent workloads contend across uplink, computation, and downlink stages. Although several approaches have been proposed, they primarily target specific tasks (e.g., video analytics) and lack support for diverse DNN tasks due to their uplink-centric designs.

This paper presents CORA, a system that serves latency-critical DNN inference requests through end-to-end coordination between the RAN and the edge server. CORA dynamically adjusts the latency budgets across stages based on each DNN task's characteristics, balancing resource demands for the radio and compute domains to mitigate contention at each stage. It then aligns the resource schedulers with these per-stage budgets, thereby enabling end-to-end coordination without requiring modifications to end hosts. We prototype and evaluate CORA on an over-the-air testbed with diverse DNN tasks. CORA serves 3.2× more requests within the latency target and reduces the 95th percentile latency by 2.1× compared to baselines.

CCS Concepts: • **Networks** → **Mobile networks**; **Network management**; *Cross-layer protocols.*

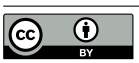
Additional Key Words and Phrases: RAN, edge-assisted DNN inference serving, cross-domain scheduling

ACM Reference Format:
Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. 2025. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proc. ACM Netw.* 3, CoNEXT4, Article 40 (December 2025), 23 pages. https://doi.org/10.1145/3768987

1 Introduction

Deep neural networks (DNNs) are increasingly essential for a wide range of mobile applications, such as immersive experiences and real-time video analytics [21, 22, 73]. Given their user-centric and interactive nature, these applications demand real-time responsiveness, e.g., 200 ms for language processing [67, 82] and 400 ms for video surveillance [33, 66]. However, mobile platforms (e.g., smart glasses [52] and headsets [8, 51]) are inherently constrained in size, power, and thermal dissipation, often making it impractical to integrate high-performance processors that can consistently meet such latency targets [9, 13, 77]. In this regard, edge-assisted DNN inference has emerged as a promising approach that offloads compute-intensive workloads to edge servers near or within the base station [5, 24, 32], thereby supporting diverse DNN tasks in mobile applications.

Authors' Contact Information: Sunghyun Jin, Seoul National University, Seoul, Republic of Korea, sunghyunjin@snu.ac.kr; Serae Kim, Seoul National University, Seoul, Republic of Korea, seraekim@snu.ac.kr; Sangtae Ha, University of Colorado Boulder, Colorado, USA, sangtae.ha@colorado.edu; Kyunghan Lee, Seoul National University, Seoul, Republic of Korea, kyunghanlee@snu.ac.kr.



ACM 2834-5509/2025/12-ART40
https://doi.org/10.1145/3768987

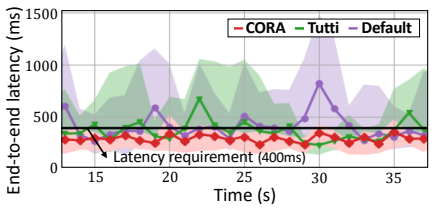


Fig. 1. End-to-end latency (mean and 95th percentile) of CORA, Tutti [77], and Default while serving diverse DNN tasks (Table 2). The uplink-centric optimization in Tutti fails to generalize across diverse DNN tasks, resulting in severe latency violations, whereas CORA consistently meets the 400 ms latency requirement.

Despite its potential, consistently meeting the end-to-end latency of diverse DNN tasks remains challenging across the entire inference process, from request to response. Measurements in commercial 5G deployments reveal substantial latency variability in DNN inference (§2). For example, the 90th percentile (P90) latency is often twice the median, with over half of the requests exceeding the 400 ms latency target [33, 66]. This variation stems from all stages of the inference pipeline, including uplink transmission, DNN execution, and downlink transmission, rather than originating from a single bottleneck. Meanwhile, diverse DNN tasks exhibit heterogeneous latency characteristics due to differences in input/output data sizes, as well as model complexity. This can degrade the user experience in modern mobile applications (e.g., XR), which often concurrently invoke multiple DNN workloads.

We argue that resource provisioning must be tailored to each DNN task across all stages of the inference pipeline.[1] Existing approaches have provided only partial solutions. To the best of our knowledge, prior works on network slicing [10, 17] and radio resource scheduling [40, 70, 77] focus on improving performance in either the uplink or the downlink, rarely considering the round-trip path of edge-assisted DNN inference. Meanwhile, server-side DNN serving systems handle diverse DNN models [18, 20, 29, 30, 61, 63], but their solutions primarily target the execution stage, with uplink and downlink transmissions—which often dominate end-to-end latency—remaining outside their scope. At the same time, most edge-assisted DNN inference systems target specific tasks such as video analytics [9, 28, 41, 81] and often rely on task-specific methods (e.g., image resolution adaptation), thereby limiting their applicability to diverse DNN tasks.

In practice, these limitations become more evident in cellular networks, where multiple users at each base station issue DNN inference requests that compete for shared resources across the radio access network (RAN) and the edge server. This concurrency complicates per-request resource provisioning, not only due to contention, but also due to heterogeneity in the computational and input/output resource demands of DNN tasks. A straightforward solution might allocate dedicated radio bandwidth and GPU resources to each request, ensuring strict end-to-end isolation. However, such isolation incurs several drawbacks. At the RAN, it limits adaptability to dynamic channel conditions, seriously degrading spectral efficiency; at the edge server, enforcing fine-grained GPU isolation requires intrusive modifications to model execution engines (e.g., PyTorch, TensorFlow).

This paper presents CORA, a system that efficiently coordinates resource scheduling across the RAN and the edge server to enable latency-critical DNN inference over cellular networks. To minimize intrusive modifications, CORA adopts a decoupled design: CORA first generates per-stage guidance that distributes the end-to-end latency budget across the uplink, computation, and downlink stages given their resource availability. It then delegates fine-grained resource scheduling to the RAN and the edge server. The primary objective of this guidance is to balance resource load across the stages, thereby allowing each radio and compute resource scheduler sufficient flexibility to optimize its performance, while collectively satisfying the end-to-end latency constraints. Ultimately, CORA aims to support diverse DNN tasks with reliable, low-latency inference over

[1]We use the term resources to refer to radio resource blocks (RBs) in the radio access network (RAN) and streaming multiprocessors (SMs) at the edge server, since we focus on coordinating these resources throughout the paper.

cellular networks. As shown in Fig. 1, which presents experimental results from an over-the-air (OTA) testbed (details in §6), our end-to-end coordination with the per-stage guidance significantly reduces tail latency and improves stability under diverse DNN tasks.

At its core, CORA enables such coordination through the following steps. For each DNN task, it first profiles and stores its input/output sizes and computation characteristics. Upon receiving a DNN inference request, CORA invokes a **DNN-aware End-to-End Planner** that derives per-stage latency and resource guidance based on the model profile and current loads (§4). Following this guidance, **Deadline-aware Resource Schedulers** (§5) allocate radio resource blocks (RBs) and GPU streaming multiprocessors (SMs). To mitigate possible deviations in uplink transmission order, potentially induced by spectral efficiency optimizations, CORA reorders inference requests before forwarding them to the edge server, without requiring changes to the model execution engines.

We implement a prototype of CORA on OpenAirInterface (OAI) [57] and evaluate its performance on a range of representative DNN tasks, including tasks based on diverse neural architectures—convolutional neural networks (CNNs) for super-resolution and image segmentation [16, 74], transformers for language processing, translation, and pose estimation [38, 62, 71][2], and multi-layer perceptrons (MLPs) for volume rendering [54]—summarized in Table 2. On the OTA testbed equipped with a GPU and commercial off-the-shelf (COTS) user equipment (UE), CORA improves the number of requests served within latency targets by 3.2× and P95 latency by 2.1× compared to baselines. Moreover, CORA incurs negligible overhead, e.g., 0.2 ms for the DNN-aware end-to-end planner and 112 B of control signaling overhead per request.

2 Motivation

Over the past decade, there has been growing interest in serving DNN tasks over cellular networks, including classification, rendering, translation, and language processing [5, 32, 49]. The community has made substantial strides in reducing DNN inference latency, including commercial deployments of edge servers such as AWS Wavelength [6] and Azure Edge Zones [53]. However, it remains uncertain whether edge-assisted DNN inference can serve as a viable platform for delivering a wide range of pre-trained DNN models in real time, as conventional server-side DNN serving systems do [20, 30]. Figure 2 illustrates how latency-critical DNN inference flows traverse the RAN and the edge server. To motivate our design, we next examine the performance under a commercial 5G deployment, revealing the challenges of serving diverse DNN tasks over cellular networks.

2.1 Measurement Setup

We examine the end-to-end DNN inference latency over commercial cellular networks, a key performance metric for DNN inference serving systems [30, 55, 66]. To this end, we use three representative DNN tasks: image segmentation [16], super resolution [74], and language processing [62] at four requests per second (RPS) (details in §6). The end-to-end latency is measured at the UE, spanning from the start of input transmission to the reception of the inference output.

5G RAN and edge server. We utilize a Google Cloud Platform n1-standard-8 instance equipped with an NVIDIA T4 GPU as the edge server. To reduce backbone latency, we select an instance located in the same metropolitan area as the UE. The UE (Samsung Galaxy Note 10) connects to the edge server over a commercial operator's sub-6GHz 5G RAN (3.6 GHz frequency, 100 MHz bandwidth). Network performance measured using `iperf` and `ping` shows an average uplink throughput of 31.2 Mbps (±14.8), downlink throughput of 424.4 Mbps (±157.6), and a round-trip time (RTT) of 34.1 ms (±9.4), where each value represents the mean and standard deviation. These measurements are consistent with recent empirical studies of commercial 5G deployments [37, 80].

[2]For auto-regressive tasks using transformers, we limit the maximum output length (e.g., 16) to meet latency constraints.

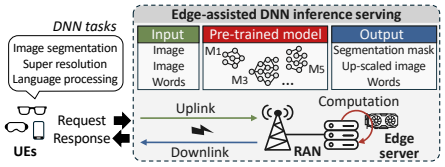


Fig. 2. Illustration of end-to-end DNN inference serving over cellular networks. The inference flow traverses the uplink, computation, and downlink stages from request to response.

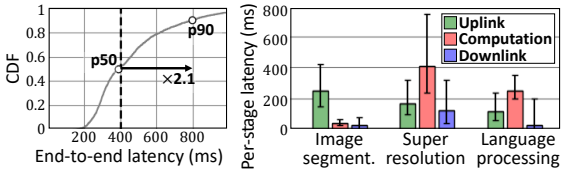


Fig. 3. Latency measurements in a commercial 5G and edge server setup. The latency breakdown reveals task-dependent variations across stages.

2.2 Measurement Results

Unreliable and long-tailed latency. Figure 3 (a) shows that the observed end-to-end latency exhibits severe variability despite the favorable network conditions. The P90 latency approaches twice the median. Moreover, the overall latency distribution frequently exceeds the response latency target, i.e., 400 ms [33, 66]. For example, the super-resolution task transmits approximately 40 KB of input and receives 120 KB of output. Given a model execution time of 221 ms on a dedicated T4 GPU and a 34 ms RTT, the expected end-to-end latency should remain well under 300 ms. Nonetheless, many observed cases exceed this expected bound.

Task-dependent distribution of tail latency. To identify where latency variation occurs, we decompose the end-to-end latency into uplink, computation, and downlink stages, as shown in Fig. 3 (b). The uplink latency is measured from the arrival of the request to the completion of input transmission, and the computation latency is measured from that point to the generation of the inference output. The downlink latency is then inferred by subtracting the uplink and computation latencies from the end-to-end latency. This breakdown reveals that tail latency arises across all three stages, with the dominant bottleneck varying by DNN task. For example, uplink latency is the primary source of latency in image segmentation. In contrast, super-resolution and language processing tasks exhibit comparable or even greater variability in the computation and downlink stages than in the uplink stage.

Notably, the minimum latency for each model remains below the 400 ms target (e.g., 372 ms for super-resolution), indicating that the observed tail latency is not fundamentally limited by physical constraints. Rather, this suggests that task-specific resource provisioning across the inference pipeline is essential for the efficient utilization of both RAN and edge server resources.

3 Design Overview

The goal of this paper is to design a reliable edge-assisted DNN inference serving system over cellular networks that provides the following design considerations.

1. **End-to-end coordination of inference pipelines**. The system must coordinate all stages of the inference pipeline—uplink transmission, DNN execution, and downlink transmission—in an end-to-end manner, accommodating heterogeneous resource demands of diverse DNN tasks.
2. **Non-intrusive integration with existing infrastructure**. Since CORA operates atop existing RAN and edge server systems, it must avoid intrusive changes to their operations. Specifically, the RAN should retain its ability to maximize spectral efficiency, and modifications to the UE stack or the edge server's model execution engine must be avoided to ensure broader applicability.

CORA meets these goals by introducing a RAN-side solution, where network operators already maintain full control, thereby enabling the practical deployment of DNN inference serving systems. Importantly, it avoids modifications to end hosts and model execution engines, while confining changes to the network operator's domain, which are self-contained. However, this design choice means that compute resource scheduling is handled internally once DNN inference begins, limiting

scheduling control to the inference-level granularity. This coarse granularity is mismatched with the radio resource scheduler, which operates at sub-millisecond timescales.

To address this gap while maintaining end-to-end coordination, CORA adopts a *guidance-delegation approach*, which consists of (1) distributing the latency budget based on the characteristics of each DNN task and the runtime resource availability at each stage (§4), and (2) delegating per-stage resource allocation to the respective radio and compute schedulers (§5). This design balances the per-stage load and prevents excessive contention at any single stage, enhancing scheduling flexibility within domains. By doing so, it allows each resource scheduler to pursue its own objectives while collectively satisfying the end-to-end latency requirements.

3.1 Challenges

End-to-end coordination under the decoupled guidance-delegation design poses several challenges.
C-1. Runtime acquisition of per-stage load. Assigning latency budgets would be straightforward if per-stage loads could be accurately estimated. However, these loads vary dynamically due to both incoming DNN inference requests and fluctuating wireless channel conditions, necessitating runtime adaptation. Although existing DNN-based simulation or prediction methods [10, 26, 44] may accurately capture channel variations, these approaches often introduce significant runtime overhead. On the compute side, even monitoring of GPU utilization incurs additional communication overhead between the edge server and the base station.
C-2. Promotion of per-stage latency and resource guidance across the domain. In the RAN, strictly allocating radio resources per inference request can lead to underutilization in dynamic channel conditions. Conversely, optimizing for spectral efficiency may disrupt the intended transmission order. In particular, reordered uplink transmissions can misalign the execution order of DNN inference at the edge server. While GPU scheduling could potentially correct this misalignment, doing so requires changes to the model execution engine and frequent coordination with the edge server—both of which introduce considerable overhead.

3.2 Key Ideas

We address these challenges with two key ideas.
K-1. Latency budget-guided scheduling map approximation. To estimate runtime loads, CORA locally constructs virtual resource scheduling maps at the base station. Our key insight is to adopt a simplifying assumption that each request utilizes resources strictly within its assigned per-stage latency budget. Under this assumption, CORA accumulates the resource demands of incoming requests over time, modeling radio resource scheduling as preemptive and compute resource scheduling as non-preemptive—reflecting their respective scheduling granularities.
K-2. In-base station latency budget enforcement. As the base station sits between the UE and the edge server, it can directly observe the completion of intermediate stages. Leveraging this visibility, CORA temporarily holds and reorders the inference requests before forwarding them to the edge server. This approach enables precise control over dispatch timing, ensuring alignment with the original latency guidance while remaining decoupled from actual GPU execution.

3.3 Workflow

CORA consists of two main components: a control path, which generates per-stage latency and resource guidance, and a data path, which allocates radio and compute resources across the RAN and the edge servers (Figure 4).

The control path employs a **DNN-aware end-to-end planner**. Before runtime, a *DNN model profiler* (§4.1) performs lightweight offline profiling for each DNN task, capturing its input/output sizes and speedup characteristics. During runtime, a *load estimator* (§4.2) leverages profiled data,

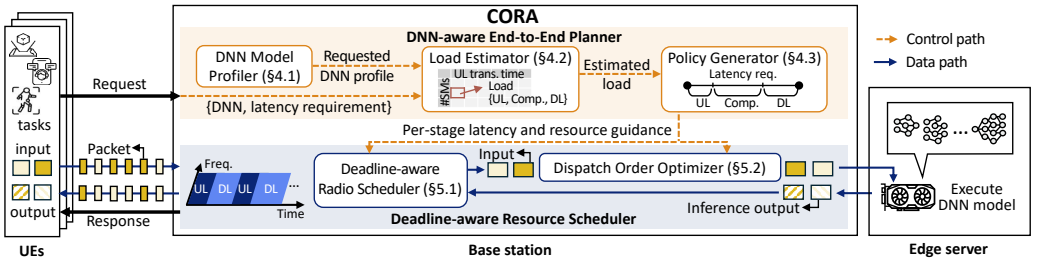


Fig. 4. System overview and workflow of CORA. It dynamically partitions the end-to-end latency based on DNN characteristics and per-stage loads, and provides per-stage guidance to resource schedulers.

historical requests, and channel quality to estimate cumulative resource demands and construct virtual resource scheduling maps. Upon receiving a DNN inference request from the UE, the *policy generator* (§4.3) selects per-stage latency budgets that balance the load across stages. These components generate per-stage latency and resource guidance that dynamically adapts to the DNN task characteristics and the availability of radio and compute resources.

To enforce the selected latency budget, CORA integrates **deadline-aware resource schedulers**. For uplink and downlink transmission stages, a *deadline-aware radio scheduler* (§5.1) allocates RBs to each request by considering both the assigned latency budget and the required throughput. For the DNN execution stage, a *dispatch order optimizer* (§5.2) reorders requests according to their assigned uplink latency budgets and enqueues them into per-SM queues based on their SM requirements, prior to forwarding them to the edge server.

4 DNN-aware End-to-End Planner

At the core of CORA, the DNN-aware end-to-end planner generates per-stage guidance, consisting of (1) target completion times for uplink transmission, DNN execution, and downlink transmission, and (2) expected radio and compute resource demands. Its goal is to minimize resource contention at any single stage, giving domain schedulers flexibility to pursue their own objectives (e.g., spectral efficiency) while jointly satisfying end-to-end latency requirements.

To estimate per-stage contention, CORA models DNN resource demands using offline profiling and runtime observations (§4.1), and builds virtual resource scheduling maps by tracking active requests (§4.2). Based on these maps, it assigns per-stage latency budgets at runtime through a lightweight procedure that combines quantization, greedy allocation, and admission policies (§4.3).

4.1 Profiling for Resource Demand Modeling

Provisioning additional RBs or SMs can reduce latency; however, the effectiveness of such provisioning varies significantly across different DNN tasks (e.g., input/output data and DNN architecture). Hence, CORA profiles the resource demands of DNN tasks to enable efficient end-to-end coordination. The profiling builds on prior work for radio resource profiling [76, 77] and compute resource modeling [18, 55, 66], and is summarized in Table 1 along with the offline and runtime parameters.

RB demand modeling. CORA estimates the radio resource demand based on the input and output data sizes of each DNN task. The input size is determined by the UE at request, while the output size is fixed for each DNN model, as it corresponds to the output layer[3]. This fixed output size typically holds for models with multiple output branches or early-exit mechanisms [34, 46, 87].

Given the input and output sizes, CORA estimates the number of required RBs based on the current channel quality, which is quantified in bytes per RB and determined by the modulation

[3]For auto-regressive tasks (e.g., language processing or translation), output size is defined as the maximum token length.

Table 1. Resource demand modeling in CORA. RB demands are derived from input/output data sizes and channel quality, where h and OH denote the protocol and the physical-layer overhead in RAN[4], respectively. SM demands are estimated using a common speedup model [11] with three parameters (d, s, c).

Resource types	Offline-profiled parameters	Runtime parameters	Demand model equations
RB demand (r^{RB})	input/output data sizes (D_I, D_O)	channel quality, bytes per RB (q)	$r^{RB} = \frac{D_{I/O} \cdot (1+h)}{q \cdot (1-OH)}$
SM demand (r^{SM})	speedup model [11] (d, s, c)	required inference time (t_{inf})	$t^{inf} = \frac{d}{r^{SM}} + s + c \cdot (r^{SM} - 1)$

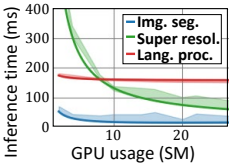


Fig. 5. Inference time modeling with speedup equation in Table 1.

order and coding rate. CORA follows the OAI implementation [57] to compute channel quality, which is compliant with 3GPP standards [1].

SM demand modeling. To capture the SM demand, CORA uses a common speedup model [11], which relates latency to SM allocation. The model has three parameters (d, s, c), representing parallel computation, sequential computation, and inter-core communication, respectively.

These parameters are obtained by fitting measured DNN inference latencies to each DNN model using least-squares regression [72], as shown in Fig. 5. For instance, profiling a super-resolution model [74] on an NVIDIA RTX 4080 takes about 10 minutes, repeating each inference (100–500 ms) 100 times across SM configurations ranging from 4 to 76 with a quantization interval of 4.

Runtime speedup model correction. CORA refines the speedup model at runtime to enhance robustness against profiling errors. The runtime DNN inference latency is measured at the base station as the duration between input forwarding to the edge server and the return of the DNN output. Although this latency includes both DNN execution time and wired transmission delay, it requires no modifications at the edge server. Based on these measurements, CORA supports two refinement approaches: (1) an exponentially weighted moving average (EWMA) for incremental adjustment, and (2) re-fitting of the speedup model using least-squares regression. These updates are triggered periodically (e.g., every 10 seconds), and the speedup model is replaced if the reconstructed model reduces the inference latency estimation error by more than half.

4.2 Per-stage Load estimation with Scheduling Map Construction

To estimate the per-stage load, CORA makes a simplifying assumption: each request utilizes resources strictly within its assigned per-stage latency budget. Based on this assumption, CORA constructs a virtual scheduling map for each specified time window by aggregating the resource demands of all active requests. The resulting map approximates the remaining resource capacity within the window—the maximum amount of resources that can be allocated to new requests without violating the latency budgets of existing ones. This estimated residual resource capacity serves as a lightweight indicator of the current load intensity at each stage.

Radio resource scheduling map. To approximate fine-grained scheduling (e.g., 0.5 ms time slots in numerology 1) in the RAN, CORA maintains two accumulated RB demand tables: a *forward table*, which cumulatively records the number of RBs used from the transmission start time onward, and a *backward table*, which accumulates RB usage in reverse from the transmission end time, as illustrated in Fig. 6 (b). By selectively combining entries from both tables, CORA constructs a virtual resource allocation map within the time window $[t_1, t_2]$, which approximates the maximum residual RB capacity without violating existing constraints. This estimated availability is visualized as the hatched RBs in Fig. 6 (b).

[4]Following [75, 77], we set $h = 0.068$ as default. For OH, we adopt 0.08 for uplink and $OH = 0.14$ for downlink, based on our testbed setting [4] (sub-6GHz NR with numerology 1 and demodulation reference signal (DMRS) type A).

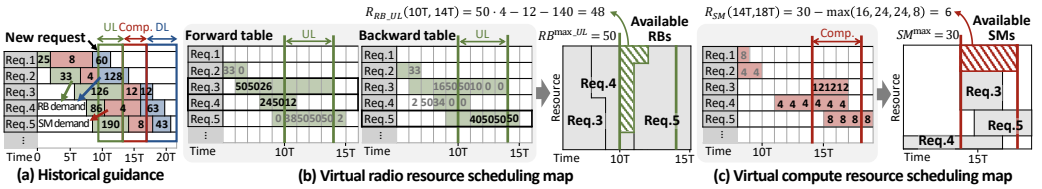


Fig. 6. Example of virtual resource scheduling map construction for the uplink stage ([10T, 14T]) and the computation stage ([14T, 18T]). The maps reflect the mismatch between fine-grained radio resource scheduling and coarse-grained compute resource scheduling.

Specifically, the estimation proceeds as follows. First, CORA identifies the relevant requests by examining their transmission intervals derived from the latency budget (i.e., $[t_1, t_2]$). Following the classification in Fig. 7, only requests in cases 2, 3, 4, and 6 that overlap with $[t_1, t_2]$ are included in the estimation. Among them, case 2 corresponds to requests whose transmission starts earlier than t_1 and are thus obtained from the forward table (e.g., Req. 3 and 4 in Fig. 6 (b)). Conversely, cases 4 and 6, whose transmission completes after t_2, are obtained from the backward table (e.g., Req. 5 in Fig. 6 (b)). The total available RBs within $[t_1, t_2]$ are computed as

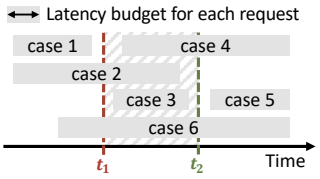


Fig. 7. Construction of the scheduling map over $[t_1, t_2]$ involves only cases 2, 3, 4, and 6, whose assigned intervals overlap with the estimation window.

$$R_{\text{RB\_UL/DL}}(t_1, t_2) = (t_2 - t_1) \cdot RB^{\text{max\_UL/DL}} - \sum_{i \in \text{case2,3}} \text{table}_\text{F}[i][t_1{:}t_2] - \sum_{i \in \text{case4,6}} \text{table}_\text{B}[i][t_1{:}t_2], \quad (1)$$

where $RB^{\text{max\_UL/DL}}$ denotes the total number of RBs per time slot[5]. table_F and table_B are the forward and backward RB demand tables, indexed by request (row) and time (column).

Compute resource scheduling map. CORA approximates scheduling at per-inference granularity when constructing the compute resource scheduling map. It builds an accumulated SM demand table, where each request is represented as a contiguous block of demand over time. The SM demand for each request is back-calculated from its DNN execution latency budget and the speedup curve.

Unlike the radio resource scheduling map, which considers fine-grained time slot accumulation, the compute resource scheduling map aggregates SM usage as contiguous demand blocks, as illustrated in Fig. 6 (c). The available number of SMs within a time interval $[t_1, t_2]$ is computed as

$$R_{\text{SM}}(t_1, t_2) = SM^{\text{max}} - \max_{t \in [t_1, t_2]} \left(\sum_i \text{table}_{\text{SM}}[i][t] \right), \quad (2)$$

where SM^{max} denotes the total number of SMs available on the GPU. table_{SM} is the SM demand table, indexed by requests (rows) and time (columns).

Runtime channel update. To maintain estimation accuracy under channel fluctuations, CORA periodically updates the accumulated RB demand tables based on each user's channel quality (e.g., every 100 ms). Since CORA operates entirely within the base station, this information can be retrieved efficiently through a shared file descriptor between CORA and the RAN processes. During each update, the RB demand of the affected requests is recomputed to reflect the new channel conditions. If the accumulated demand exceeds $RB^{\text{max\_UL/DL}}$, the excess RBs are deferred and appended beyond the original latency budget interval.

[5]In time-division duplexing (TDD), each time slot is assigned as uplink or downlink. We approximate the number of radio resource blocks (RBs) using the configured uplink/downlink ratio, instead of inspecting each slot assignment.

Design considerations. The per-stage load estimation is a lightweight approximation of domain-wise schedulers to enable efficient runtime execution, rather than exact emulation. While this approximation assumes that each request uses resources strictly within its assigned latency budget, the system remains effective even under heavy loads where this assumption may occasionally be violated (§6.1). This robustness stems from the fact that real-time DNN inference workloads are completed within a finite time window; thus, the error does not grow unbounded over time.

While runtime feedback from domain schedulers, such as RB or SM utilization per time slot, can refine the estimation, it may lead to frequent cross-domain interactions and additional overhead. Thus, we leave the practical design of such a fine-grained feedback loop across RAN and edge servers for future work, while maintaining lightweight estimation as a primary design focus.

4.3 Runtime Per-stage Latency Budget Assignment

As per-stage loads fluctuate with the active DNN inference requests and channel quality, CORA adopts an online greedy policy that runs upon each request arrival. To enable efficient planning, CORA limits the search space to a quantized set of latency budget candidates, achieving polynomial-time complexity in both the number of concurrent requests and the number of quantization levels.

Latency budget allocation strategy. CORA assigns latency budgets to minimize contention at any single stage. To achieve this strategy at runtime, it employs a Lagrangian relaxation framework [14, 64], which transforms resource constraints into optimization objectives, as formulated below.

Consider a request i arriving at time $t_{i,\text{req}}$, with an end-to-end latency deadline $t_{i,\max}$. The latency budget selection policy assigns $\boldsymbol{t}_i = (t_{i,\text{ul}}, t_{i,\text{comp}})$, corresponding to the completion times of uplink transmission and DNN execution, respectively. The per-stage latency budgets are then defined as the intervals between adjacent decision points: uplink $(t_{i,\text{req}}, t_{i,\text{ul}})$, computation $(t_{i,\text{ul}}, t_{i,\text{comp}})$, and downlink $(t_{i,\text{comp}}, t_{i,\max})$. Given these intervals, CORA estimates the per-stage load using the virtual scheduling maps $R_{\text{RB\_UL/DL}}$ and R_{SM}, as defined in Eqs. 1 and 2. Based on these estimations, it determines the optimal latency budget $\boldsymbol{t}_i$ by solving the following optimization problem, which subsequently guides the domain schedulers:

$$\max_{\boldsymbol{t}_i} \sum_i \Big(\lambda_{i,\text{ul}} \cdot R_{\text{RB\_UL}}(t_{i,\text{req}}, t_{i,\text{ul}}) + \lambda_{i,\text{comp}} \cdot R_{\text{SM}}(t_{i,\text{ul}}, t_{i,\text{comp}}) + \lambda_{i,\text{dl}} \cdot R_{\text{RB\_DL}}(t_{i,\text{comp}}, t_{i,\max}) \Big), \quad (3)$$

$$\text{s.t.} \quad t_{i,\text{req}} \le t_{i,\text{ul}} \le t_{i,\text{comp}} \le t_{i,\max}, \quad \forall i.$$

Here, $\lambda_{i,\text{ul}}$, $\lambda_{i,\text{comp}}$, and $\lambda_{i,\text{dl}}$ denote the Lagrangian multipliers associated with the uplink, compute, and downlink resource constraints, respectively. By default, these weights are set to $(1/RB^{\max\_\text{UL}}, 1/SM^{\max}, 1/RB^{\max\_\text{DL}})$, a normalized constant to balance domains. However, they can be adjusted to reflect the relative importance of each resource if necessary.

The selected latency budgets are then passed to the domain schedulers (§5), which conduct fine-grained resource allocations. They can independently manage fairness, DNN inference dispatch timing, or task priority as long as the guided latency budgets are satisfied.

Online greedy policy. The optimization problem above is computationally intractable, as the number of possible completion time combinations grows exponentially with the number of concurrent requests. Furthermore, the optimal solution depends on time-varying channel quality and unpredictable arrivals of future DNN tasks, making offline planning impractical. To address this, CORA employs an online greedy policy that approximates the solution at runtime. Upon the arrival of the i-th request, $\boldsymbol{t}_i$ is selected to maximize the weighted sum of per-stage residual resources, as estimated using the virtual scheduling maps constructed from the previous $i-1$ requests.

Admission control policy. While the end-to-end planner aims to balance resource demands across stages, the cumulative load may still exceed resource capacities under conditions of high contention or sudden channel degradation. To handle such cases, CORA operates by default in an

admission-control mode, where a request is accepted only if the resulting latency budget assignment yields strictly positive residual capacities for both $R_{\text{RB\_UL/DL}}$ and R_{SM}. In a best-effort mode, CORA turns off admission control and admits all requests, even if the resulting residual capacities are negative, while still performing latency budget optimization as formulated in Eq. 3.

Quantized policy candidate lookup. To reduce the runtime complexity of the planning module, CORA does not directly solve the optimization problem in Eqn. 3. Instead, it restricts its search to a quantized set of latency budget candidates. Quantization is applied along two dimensions: (1) $t_{i,\text{ul}}$, the uplink transmission completion time, and (2) r^{SM}, the number of allocated GPU SMs. CORA adopts r^{SM} rather than $t_{i,\text{comp}}$, to reflect the discrete nature of SM allocation and to enable precomputation of the corresponding DNN execution times (see Table 1).

Time complexity. The resulting time complexity is $O(Q_t^2 \cdot Q_c \cdot N_r)$, where Q_t and Q_c denote the number of quantization bins for $t_{i,\text{ul}}$ and r^{SM}, respectively, and N_r is the number of concurrent requests. This arises from evaluating $Q_t \cdot Q_c$ candidate combinations and querying a virtual resource scheduling map of size $Q_t \times N_r$ for each. We empirically evaluate the execution time on an Intel Core i9-10940X CPU. With a latency budget quantization of 20 ms and a scheduling horizon of 560 ms ($Q_t = 28$), and GPU SMs quantized in 4 SM steps ($Q_c = 19$ under RTX 4080 with $SM^{\max} = 76$), the execution time remains under 0.5 ms even when handling more than 600 concurrent requests, as shown in Fig. 8.

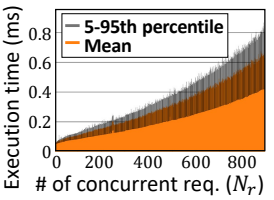


Fig. 8. Execution time of the DNN-aware end-to-end planning module.

5 Deadline-aware Resource Scheduler

To enable practical deployment, CORA's deadline-aware resource scheduler operates on the RAN side, without requiring any modifications to end hosts such as UE stacks or model execution engines. Figure 9 illustrates the scheduling workflow, consisting of a deadline-aware radio scheduler (§5.1) and a dispatch order optimizer (§5.2), allocating radio and compute resources for each request based on the latency and resource guidance assigned by the DNN-aware end-to-end planner.

5.1 Deadline-aware Radio Scheduler

CORA performs deadline-aware radio scheduling by registering uplink/downlink guidance for each inference request (②/⑥), allocating radio resources based on urgency and spectral efficiency (③/⑦), and continuously tracking the transmission progress of each request (④/⑧).

Uplink and downlink guidance registration. Upon each inference request, the planner sends a guidance registration containing the data size, latency budget, and periodically measured channel quality (sampled every 100ms). This registration is triggered at the beginning of both the uplink and downlink stages via a UNIX socket interface connecting the planner and the gNB process (e.g., the `nr-softmodem` process in OAI [57]).

To avoid interference with the existing threads in the gNB process, CORA introduces a dedicated thread for guidance registration, referred to as the `Registration and Monitoring Thread` in Fig. 9. The received information is stored in the internal request table (Fig. 10). While CORA adopts low-overhead UNIX sockets to minimize runtime overhead, other frameworks, such as ZMQ-based real-time RAN Intelligent Controllers (RICs) [42], can also be supported.

Deadline-aware RB provision. CORA builds its RB provision on deficit round robin (DRR), a widely adopted scheduling algorithm that supports multiple flows with diverse requirements [39, 65]. However, a direct adoption of DRR is unsuitable due to two key reasons. First, prioritizing solely based on throughput can result in latency budget violations, as the input/output data sizes of DNN tasks often vary by more than an order of magnitude. Second, while maintaining high

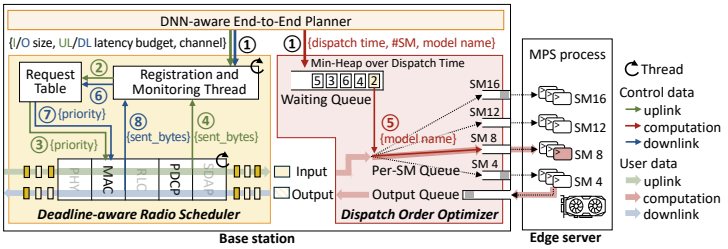


Fig. 9. Workflow of control and user data interactions that enforce per-stage guidance between the RAN and the edge server.

```
typedef struct uplink_downlink_guidance {
    int update_timestamp;

    int input_data_size;
    int uplink_latency_budget;
    int uplink_channel;
    int uplink_sent_bytes;

    int output_data_size;
    int downlink_latency_budget;
    int downlink_channel;
    int downlink_sent_bytes;
};
```

Fig. 10. Structure for the request table in the gNB process.

spectral efficiency is essential, naïvely adapting to fluctuating channel conditions may distort load estimations, since channel quality is only updated periodically (e.g., every 100 ms) for the planner. To address these issues, CORA defines the scheduling priority of request i as,

$$P_i = \left(\frac{\mathrm{B}_i^{\mathrm{req}} - \mathrm{B}_i^{\mathrm{cur}}}{\mathrm{B}_i^{\mathrm{req}}} \cdot \frac{q_i^{\mathrm{cur}}}{q_i^{\mathrm{req}}} \right) + 2^{\max(10 - T_i^{\mathrm{cur}}, 0)}, \tag{4}$$

where $\mathrm{B}_i^{\mathrm{req}}$ is the required throughput, computed from the data size and the latency budget, and $\mathrm{B}_i^{\mathrm{cur}}$ is the observed throughput based on elapsed time and sent bytes. q_i^{req} (`uplink/downlink_channel` in Fig. 10) and q_i^{cur} represent the channel quality at planning time and at runtime, respectively, and T_i^{cur} is the remaining time until the transmission latency budget.

This priority function balances adherence to the guidance with spectral efficiency. To account for variability in input/output sizes and deviations between q_i^{req} and q_i^{cur}, both the throughput deficit and channel quality are normalized. This design also prevents starvation of best-effort flows by capping P_i until the latency budget nears expiration (§6.4).

Per-request transmission monitoring. The calculation of P_i involves tracking the remaining latency budgets and transmitted bytes for each DNN inference request. First, CORA uses the system frame number (SFN) and slot index of the gNB to compute the remaining latency budgets. Upon registration, it records a timestamp (`update_timestamp` in Fig. 10). The elapsed time is then obtained as the difference between the current time and the recorded timestamp, and subtracted from the assigned latency budget to yield the remaining time.[6]

Second, CORA maintains a per-flow counter of transmitted bytes (`sent_bytes` in Fig. 10). To distinguish concurrent requests from the same UE, it identifies flows using IP/port tuples in packet headers. In the downlink, CORA maintains separate radio link control (RLC) sub-layer queues for each flow and tracks allocated bytes, following the method in [40]. In the uplink, it counts IP packet sizes after deciphering at the packet data convergence protocol (PDCP) sub-layer, thus enabling per-flow tracking without modifications to the UE-side RAN stack.[7]

5.2 Dispatch Order Optimizer

After the uplink transmission, CORA holds incoming request inputs in a waiting queue and assigns them to per-SM queues based on their per-request latency budgets and SM allocations. The requests are then forwarded to the edge server in the planned execution order by the guidance, thereby decoupling the dispatching logic from the edge server.

[6]This approach remains feasible given the bounded SFN range (i.e., 1024, or 10.24 seconds), since the expected uplink/downlink transmission latency budgets are well within a few hundred milliseconds.

[7]Alternatively, per-flow tracking can be implemented at the user plane function (UPF) by assigning each flow to a dedicated data radio bearer (DRB) [15, 36] and tracking the allocated transport block size (TBS) per DRB.

Request order reconciliation. The right part of Fig. 9 depicts the workflow of request order reconciliation. After the planning phase, the guidance for each DNN inference request (i.e., its latency budgets and SM requirements) is enqueued into the waiting queue implemented as a min-heap, ordered by uplink transmission deadlines $t_{i,\text{ul}}$ (①). Each request is initially marked as uncompleted and is updated to completed once its input transmission finishes. Among the completed requests, those whose deadlines $t_{i,\text{ul}}$ have passed are dispatched to per-SM queues according to their SM requirements (⑤). Subsequently, these requests are forwarded to the edge server for DNN execution.

To improve GPU utilization, CORA relaxes the dispatch condition in two cases. First, if the request at the head of the queue remains incomplete after its uplink latency budget has expired, CORA checks the subsequent request in the queue and dispatches it if its latency budget has also expired. Second, CORA allows early dispatching of the frontmost completed request, even before its $t_{i,\text{ul}}$, to avoid idle GPU time. This reconciliation process has a worst-case complexity of $O(N_r \cdot \log N_r)$, where N_r is the number of concurrent requests.

Transparent edge server-side DNN execution. Once dispatched into per-SM queues, each request is forwarded to a dedicated edge-side process (`MPS process` in Fig. 9) that is exclusively assigned to the per-queue based on SM allocation. Thus, each MPS process receives requests only from its associated queue and directly executes DNN inference without any additional scheduling logic. These processes operate under the NVIDIA MPS control daemon [19], which restricts the maximum percentage of SMs available per process. Since SM allocation is determined at process launch, they are pre-launched according to the configured SM quantization. Through this design, CORA only needs to send the model name and input data to the edge server, thereby ensuring compatibility with diverse model execution engines while enforcing per-request dispatch timing and SM allocation guidance.

6 Evaluation

Implementation. We implemented CORA using Python and C. On the base station side, the DNN-aware end-to-end planner and the dispatch order optimizer are implemented in Python (2.4K lines) and compiled with Numba [43] for efficient array operations. These modules operate on a 20 ms time quantization interval and a 4 SM quantization. For the deadline-aware radio scheduler, we build on top of the OAI user-plane stack (Layer 2) with over 1.3K lines of C code. To enable the concurrent execution of `nr-softmodem` from OAI and CORA on the same host machine, we isolate CPU cores, assigning {0,2,4,6,8,10,12} to OAI and {1,3,5} to CORA.

At the edge server, DNN inference is performed using PyTorch [7] with models preloaded onto the GPU to eliminate loading overhead, following common practices in prior work [9, 28, 56, 77].

OTA testbed setup. We build an end-to-end 5G network testbed using OAI [57] and Open5GS [60] as shown in Fig. 11. The testbed consists of six Google Pixel 6 and one Pixel 5 devices with programmable SIM cards (sysmoISIM-SJA2 [68], OpenCells-SIM [59]), a USRP X310 RF frontend, and an Intel Core i9-10940X host machine. The base station operates on Band 41 (2574 MHz, 40 MHz bandwidth) under numerology 1 and a TDD configuration of "DDDDDDDSUU", as typical commercial 5G deployments [31]. For DNN inference, we use an edge server equipped with an NVIDIA RTX 4080 GPU, located in the same room.

To evaluate realistic wireless channel conditions, we use a commercial 5G channel trace collected in a metropolitan area using XCAL [2], assigning each UE a randomly chosen starting point. We evaluate performance under diverse conditions, including different TDD configurations, GPU settings, and real-world indoor scenarios without channel traces (§6.3).

DNN workloads. We evaluate CORA using six representative DNN tasks, as summarized in Table 2. All models are executed in half-precision (FP16). We use a batch size of 1 across all experiments,

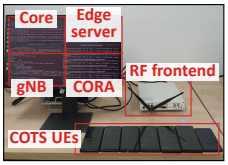


Fig. 11. Testbed setup.

Table 2. List of DNN tasks used in the evaluation.

DNN task	Input data (dimension)	Output data (dimension)
Image segmentation (M1) [16]	Image (3×196×196)	Segmentation mask (1×196×196)
Pose estimation (M2) [38]	Two images (2×3×112×112)	3D pose vector (16×3)
Language processing (M3) [62]	8 random words (1×8)	Up to 16 words (1×16)
Translation (M4) [71]	8 random words (1×8)	Up to 16 words (1×16)
Super resolution (M5) [74]	Image (3×112×112)	Up-scaled image (3×224×224)
Volume rendering (M6) [54]	Ray origins and directions (1×3 vector, 3×3 matrix)	Rendered image (3×196×196)

following common practice in real-time DNN inference serving [55, 63, 69, 77].

Each UE issues inference requests following a Poisson arrival process with a mean arrival rate of $\lambda = 4$ RPS, and the requested DNN task is randomly selected from the six tasks. Unless otherwise specified, the latency requirement is fixed at 400 ms, following prior studies [33, 66], and each experiment consists of over 3,000 inference requests.

Evaluation metrics. We evaluate CORA using two main metrics: (1) **End-to-end latency**, measured as the time from request at the UE to output reception; and (2) **Goodput**, defined as the number of inference requests completed within their latency requirements, reflecting the system's resource efficiency.

Baselines. We evaluate three baseline systems and one variant of CORA:

- **CORA-BE (best-effort)**: A variant of CORA without admission control.
- **Default**: A system using the default proportional fair (PF) scheduler in OAI and the default GPU scheduler in a first-in-first-out (FIFO) manner.
- **Tutti** [77]: A real-time video analytics system that allocates uplink resources under a fixed latency budget[8]. Following the original design, the uplink deadline is set to 200 ms (i.e., half of the 400 ms end-to-end latency requirement), and DNN execution follows FIFO order. For RB calculation, we modify Tutti to use the actual input data size, consistent with CORA, instead of its original frame-size predictions.
- **SEP (separated)**: A system with statically partitioned latency budgets: 200 ms for uplink, 100 ms for computation, and 100 ms for downlink. We adopt Tutti for uplink and extend it to downlink. DNN execution follows CORA's speedup model and SM allocation with a 100 ms latency budget.

Summary. The highlights of our evaluation are as follows:

- CORA improves goodput by 3.2× and reduces P95 end-to-end latency by 2.1× on average over the baselines (§6.1). It achieves these gains while incurring minimal overhead (§6.2).
- CORA adapts to diverse TDD, GPU, and channel conditions, achieving 1.79–3.74× higher goodput and 1.51–2.71× lower P95 end-to-end latency in each scenario, compared to the baselines (§6.3).
- CORA's deadline-aware resource scheduler effectively enforces the latency budget, while its radio resource scheduler maintains spectral efficiency and fairness with background traffic (§6.4).

6.1 CORA improves end-to-end performance

We first evaluate whether and by how much CORA improves the end-to-end performance. Figure 12 (a) and (b) show the goodput and end-to-end latency (P95 shown as the upper whisker), respectively, as the number of concurrent requests increases. CORA, CORA-BE, and SEP consistently outperform Tutti and Default, which consider only individual pipeline stages. As the load

[8]Most video analytics systems [28, 41, 79, 81] use image/video-specific adaptations (e.g., encoding rate) that do not generalize to other tasks (M3–M6). Tutti instead provides a generalizable uplink radio resource scheduling that accounts for both data size and latency budget, making it the most suitable baseline.

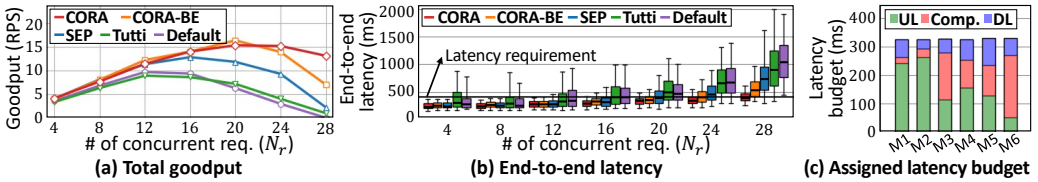


Fig. 12. Performance under varying numbers of concurrent requests. CORA and CORA-BE achieve higher goodput and lower latency than the baselines by dynamically adapting latency budgets.

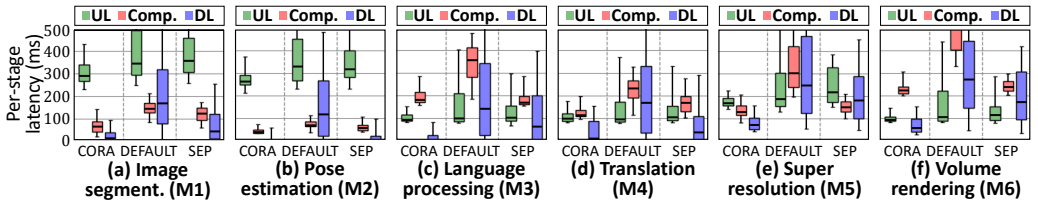


Fig. 13. Latency breakdown of CORA, DEFAULT, and SEP under 24 concurrent requests (N_r). The figure shows the median (line), interquantile range (P25–P75), and whiskers (P5–P95). CORA achieves lower variability by operating in an end-to-end manner, with domain schedulers enforcing this behavior (see Fig. 18).

increases, CORA increases its advantage, while SEP begins to violate the 400 ms latency requirement beyond $N_r = 12$. This is because SEP fails to efficiently utilize cross-domain resources due to its static allocation of latency budget. In contrast, CORA leverages the DNN-aware end-to-end planner that dynamically adapts to per-stage load variations and channel quality, ensuring balanced resource utilization across the inference pipeline.

Heterogeneous DNN tasks. Figure 12 (c) shows the average latency budget assigned for each DNN task, illustrating how CORA accommodates heterogeneous resource demands. The planner considers input/output data sizes, computational complexity, and parallelization characteristics (e.g., allocating 4 SMs to language processing (M3) with low parallelism, and 16 SMs to super-resolution (M5) with high parallelism). In the case of the super-resolution task (M5), although the output size is larger than the input, the planner assigns smaller latency budgets to the downlink stage. This assignment reflects resource capacity at each stage when allocating per-stage budgets. Specifically, under the TDD configuration in our evaluation, uplink slots are more constrained than downlink, prompting CORA to allocate a larger portion of the latency budget to the uplink stage.

Latency breakdown. Figure 13 shows the latency of each stage (uplink transmission, DNN execution, and downlink transmission) under heavy load ($N_r = 24$). As analyzed above, both CORA and SEP achieve lower latency than the Default, but SEP suffers from higher variability. In particular, downlink transmission time (Fig. 13 (a), (c), (e), and (f)) exhibits significantly larger variance for SEP than for CORA. This arises because SEP allocates a fixed 100 ms budget regardless of output size; when multiple large outputs simultaneously exceed their deadlines, SEP prioritizes all overdue flows, ultimately resembling a round-robin scheduling approach.

Performance over time. Figure 14 (a) and (b) show the CDF and P95 of end-to-end latency, where P95 latency is sampled every second under $N_r = 16$ and 24, respectively. CORA achieves significantly more stable tail latency than the baselines by dynamically adjusting its latency budgets and resource scheduling in response to channel variations and per-stage load fluctuations. Figure 14 (c) presents the admit ratio over time, illustrating how CORA adapts admission decisions based on radio and compute resource availability to sustain tail latency stability.

Per-stage load estimation. Figure 15 shows the performance of the per-stage load estimation. We evaluate its accuracy by comparing the estimator's predicted violations under CORA-BE with

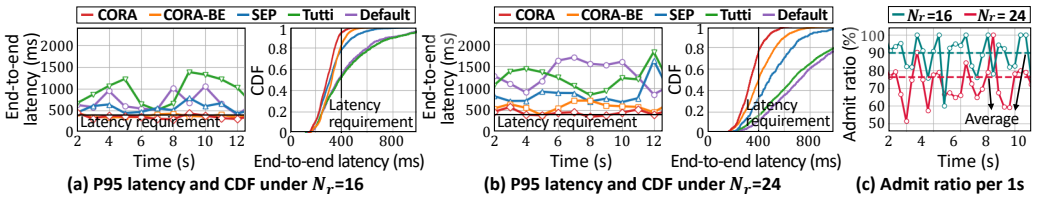


Fig. 14. P95 and distribution of end-to-end latency and CORA's admission ratio over time (sampled every second), under 16 and 24 concurrent requests (N_r). CORA maintains low tail latency by selectively admitting requests based on channel quality and system load.

the actual latency outcomes. While under heavy load ($N_r = 24$), the assumption that schedulers strictly adhere to the assigned latency budgets may not always hold, the estimator achieves an F1 score of 0.72, with both precision and recall exceeding 0.7. Under a lighter load ($N_r = 16$), the F1 score improves to 0.89. However, under the highest load ($N_r = 28$), precision drops to 0.5. This degradation stems from excessive resource demands that spill over beyond the latency budget and into the time window during the construction of the virtual scheduling maps, particularly from the requests classified as case 1 and 5 in Fig. 7.

6.2 CORA incurs low overhead

We evaluate the runtime overhead of CORA under a high-load setting with $N_r = 24$. Table 3 summarizes the overhead introduced by the DNN-aware end-to-end planner, which reflects the pure cross-domain coordination cost. The planning procedures, including per-stage load estimation, take only 0.18 ms, even when running on just three CPU cores.[9] The accumulated resource demand tables and virtual resource scheduling maps occupy approximately 40 KB, as CORA maintains entries only for active requests and removes expired ones. Although both selection time and memory usage increase with the number of concurrent requests (N_r), the overhead remains modest; for example, at $N_r = 16$, the planning time and virtual map size are 0.17 ms and 23 KB, respectively.

Next, we measure the per-request registration overhead between CORA and the gNB. Table 4 shows that the message sizes and one-way latency remain small and stable. Finally, Table 5 presents the runtime CPU and memory overhead introduced on the gNB process. Including its additional thread and per-flow tracking procedures, CORA increases CPU usage by only 1.6% and memory consumption by 9 MB. These low overheads suggest that CORA can be practically deployed within the base station node.

6.3 CORA adapts to variability

Different latency requirements. Figure 16 presents the end-to-end latency of CORA and CORA-BE under $N_r = 24$, where each DNN inference request is randomly assigned a latency requirement between 400 and 800 ms. The results show that CORA effectively prioritizes requests with tighter latency constraints, underscoring its potential for value-based or price-based prioritization in future DNN service deployments.

Different bottleneck stages. We evaluate the performance of CORA under different bottleneck stages. Figure 17 (a) presents results under a different TDD configuration ("DDDDSUUUUU") and a constrained GPU setup (NVIDIA RTX 3070). CORA achieves at least 1.79× and 3.74× higher goodput, respectively, compared to the best-performing baseline. Notably, SEP performs poorly under GPU constraints, as it oversubscribes GPU resources to enforce a fixed 100 ms execution time. This performance gap stems from CORA's ability to adapt latency budgets to per-stage load, as

[9]The execution time in Fig. 8 is measured using all 14 CPU cores of the Intel Core i9-10940X.

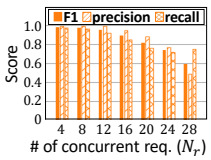


Fig. 15. Per-stage load estimation under CORA-BE.

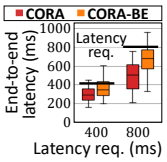


Fig. 16. Latency requirements (400 ms/800 ms).

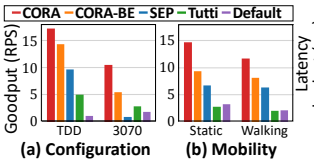


Fig. 17. Goodput across varying configurations and mobility scenarios under $N_r = 24$. CORA adjusts latency budget to balance per-stage loads under different TDD and GPU configurations.

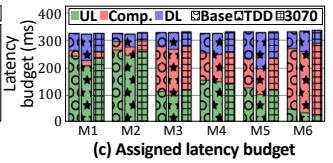


Table 3. DNN-aware end-to-end planning module overhead.

Latency budget selection time (ms)	Virtual scheduling map size (KB)
0.18±0.03	39.7±18.0

Table 4. Per-request registration overhead at the RAN.

Per-request meta data (B)	One-way latency (ms)
112B per request	0.06±0.07

Table 5. Runtime overhead on gNB.

	CPU usage (%)	Memory usage (MB)
Default	132.2±10.5	1332.7
CORA	133.8±9.5	1341.6

shown in Fig. 17 (c). Under the TDD configuration, which provides more uplink but fewer downlink slots, CORA reduces the uplink latency budget by 8–23% and increases the downlink budget by 6–56%. With limited GPU resources, it also extends the compute latency budget by 6–18%.

Different channel conditions. To assess performance under diverse wireless channel conditions, we conduct OTA experiments without channel traces. In the static scenario, UEs are placed within 1 m of the RF frontend, and in the mobility scenario, UEs move at 1.5 m/s across a 10 m×5 m room while connected to a single base station. As shown in Fig. 17 (b), CORA improves goodput for both static and walking scenarios, achieving 2.18× and 1.85× the throughput of the best-performing baseline, respectively. These gains are enabled by runtime adaptation to channel conditions, where CORA updates the channel quality every 100 ms.

6.4 Microbenchmarks

Latency budget enforcement under varying loads. We evaluate adherence to latency budgets by measuring deviations between planned stage boundaries ($t_{\text{ul}}, t_{\text{comp}}$) and actual completion times. A 40 ms tolerance—twice the quantization step—is applied to account for granularity without affecting the budgets. Figure 18 (a) and (b) show the results under different loads ($N_r = 16$ and 24). Under lighter load, CORA and CORA-BE attain 95% and 87% adherence within 40 ms, respectively. Under heavier load, CORA indeed exhibits a larger gap between the uplink and DNN execution stages. The observed deviations arise from low-level GPU variability, such as memory/cache management and the behavior of hardware CUDA schedulers; mitigating such variability would require deep modifications, for example, kernel- or compiler-level redesigns [30, 55, 67]. Although we avoid such intrusive modifications to preserve deployability for MNOs, CORA can interoperate with these advanced systems (as discussed in §8).

Impact of channel quality update. To analyze the impact of runtime adaptations, we fix the channel quality at $q = 50$ bytes per RB for all UEs in the end-to-end planner. Figure 18 (c) shows that the uplink scheduling performance of CORA and CORA-BE converges, unlike in Fig. 18 (a). This occurs because the un-updated channel causes a misalignment between modeled and actual RB demand, rendering runtime adaptation ineffective. These results highlight that offline profiling and runtime channel updates must work in tandem to plan latency budgets and perform admission control effectively.

RAN objectives: Spectral efficiency. Figure 19 reports uplink and downlink spectral efficiency, measured during active time slots under TDD. For a fair comparison of radio resource schedulers, we report spectral efficiency with CORA-BE, since admission control alters traffic loads. CORA-BE achieves higher uplink efficiency than Tutti, empowered by adaptively assigned uplink budgets.

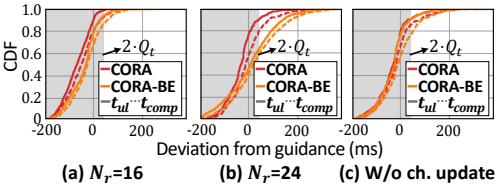


Fig. 18. Latency enforcement by domain schedulers under varying loads (a, b) and no channel updates (c).

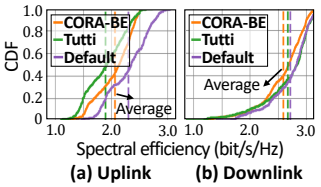


Fig. 19. Distribution and average values of spectral efficiency.

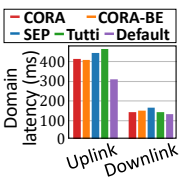


Fig. 20. Fairness w/ background traffic.

This increases flexibility for domain schedulers, yielding an average of 2.05 bit/s/Hz, compared to Tutti's 1.89 bit/s/Hz. For downlink, CORA achieves an average spectral efficiency of 2.59 bit/s/Hz, 5% lower than the Default (2.72 bit/s/Hz). Note that Tutti also uses the PF scheduler for downlink.

RAN objectives: Fairness. To assess fair coexistence with background traffic, we measure uplink file upload and downlink web page load times using a Flask-based web server, following the methodology of [77]. Each transaction involves a 196×196 image. The experiment includes four UEs generating inference requests and two UEs for background traffic. As shown in Fig. 20, compared to the default PF scheduler, CORA introduces only 94–104 ms of additional uplink latency, whereas Tutti and SEP incur 134–154 ms. On the downlink, CORA increases latency by 12–18 ms, while SEP adds 33 ms. This low impact arises because CORA prioritizes only when necessary with an end-to-end view, capping priorities of inference flows until their latency budgets near expiration.

7 Related Work

Latency-critical service assurance in cellular networks. Several works have aimed to support latency-critical services such as web loading or video analytics over cellular networks [40, 70, 77]. These approaches are effective in achieving their respective service goals. Still, from the perspective of edge-assisted DNN inference serving, they address only a single stage of the end-to-end pipeline, focusing on either the uplink [70, 77] or the downlink [40] stage. Another line of work explores RAN slicing [10, 17, 47, 48, 83], which partitions radio resources to meet service-specific performance requirements. These methods can complement CORA to isolate real-time DNN inference flows, thereby enabling efficient scheduling without degrading the performance of other traffic.

Model serving systems in the cloud. A variety of systems have been proposed to improve the efficacy of DNN inference serving, reducing inference latency [18, 20, 29, 30, 55] and cost [12, 45, 61, 63, 85]. For example, Clipper [20], Swayam [29], and INFaaS [63] adapt batching policies, select model variants, or optimize auto-scaling policies to meet latency targets. Clockwork [30] exploits the predictability of DNN inference to provide accurate latency guarantees. Building on this predictability, gpulet [18] and μ-Serve [61] enable fine-grained spatial multiplexing and batching. While they provide useful insights, inference serving over cellular networks hinges on uplink and downlink transmission, often over half the total latency (see Fig. 3 (b)). They also vary with data size and channel conditions, necessitating end-to-end coordination of the RAN and the edge server.

End host adaptations for edge-assisted DNN inference. In offloaded inference over wireless links, prior systems often adapt either the UE [9, 41, 79, 84] or server-side configurations [50, 56]. Entro [41] and ARMA [81], for instance, dynamically adjust video encoding rates based on network bandwidth; ARMA further employs a real-time RAN Intelligent Controller (RT-RIC) [42] to control uplink radio resource scheduling. In contrast, Jellyfish [56] and RAMSIS [50] select appropriate model variants after the uplink transmission completes. Unlike these approaches, which require task-dependent modifications to mobile applications or rely on pre-installed model variants, the

guidance-delegation approach of CORA remains non-intrusive to both the UE stack and the server-side model execution engines, thereby ensuring broader applicability in cellular deployments.

8 Discussion

Integration with model serving systems in the cloud. There has been extensive research on reducing inference latency in the cloud through techniques such as kernel-level scheduling [30, 55, 67], dynamic batching [18, 20], and memory access optimization [27]. While CORA primarily utilizes spatial multiplexing via NVIDIA MPS [19], its design can integrate with these cloud-side optimizations in two key aspects. First, CORA assigns per-request latency budgets via end-to-end planning, which cloud-based GPU schedulers can leverage to prioritize or schedule DNN executions without requiring access to or monitoring of the RAN. Second, CORA does not require modifications to model execution engines, as long as they support per-inference latency controls, which are already supported by modern inference serving frameworks [18, 30, 55, 67]. Although additional offline profiling may be needed to tune specific configurations, CORA imposes no constraints on the internal mechanisms of the model execution engines.

Offline DNN model profiling. Similar to existing inference serving systems [18, 33, 63, 66], CORA performs offline profiling but also compensates for runtime deviations. However, two cases necessitate additional profiling efforts. First, whenever a new DNN task is introduced or the GPU hardware is changed, the system must re-profile the task. The profiling process is relatively lightweight (e.g., 10 minutes per task), but its cost can be further reduced by adopting estimation techniques [25, 86]. Second, for auto-regressive models such as large multimodal models (LMMs), inference time varies with the number of generated tokens. CORA currently conservatively assumes the maximum token length, but a more accurate prediction would enable better resource efficiency. While recent work [3, 35] has addressed token-length prediction, these methods are not directly applicable to edge-assisted inference, where input data is not fully available until the uplink transmission completes. We believe that this limitation could potentially be addressed by predicting token length from partial or incomplete inputs, which we plan to explore in future work.

Adaptability to multi-cell and multi-edge scenarios. In vRAN and O-RAN deployments, a centralized (CU) or distributed unit (DU), where CORA can be deployed, may be connected to multiple radio units (RUs) [23, 58], and dense edge server deployments often connect a base station to numerous servers [78]. We anticipate that such multi-connectivity creates opportunities for improving efficiency by steering inference traffic toward underutilized nodes. However, this requires expanding the virtual resource and candidate lookup tables with additional dimensions, which in turn increases the complexity of end-to-end planning. Nevertheless, the end-to-end planning processes of CORA involve only lightweight vector and table operations; thus, we believe the overhead of expanded tables is manageable.

9 Conclusion

Latency-critical DNN inference is becoming increasingly essential to modern mobile applications. This paper presents CORA, an edge-assisted model serving system that efficiently handles diverse DNN tasks under latency constraints. CORA dynamically adjusts per-stage latency budgets by accommodating both the heterogeneous resource demands of each DNN task and runtime variations across the end-to-end inference pipeline. We implement and evaluate CORA through over-the-air experiments, demonstrating that end-to-end coordination between the RAN and the edge server significantly reduces tail latency and improves overall goodput. Moreover, CORA operates primarily at the RAN side, enabling seamless deployment within existing cellular infrastructure without requiring modifications to user devices or model execution engines.

Acknowledgments

We sincerely thank our anonymous shepherd and the reviewers for their valuable feedback and guidance. This work was supported by the National Research Foundation of Korea (NRF) (RS-2022-NR070834) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grants funded by the Korea government (MSIT) (RS-2024-00405128, RS-2024-00418784, RS-2024-00398157). Kyunghan Lee and Sangtae Ha are co-corresponding authors of this work.

References

[1] 3GPP TS 38.306. 2025. NR; User Equipment (UE) radio access capabilities (Release 18).
[2] ACCUVER. 2024. XCAL: PC based advanced 5G network optimization solution. https://accuver.com/sub/products/view.php?idx=6 Accessed: 2025-06-05.
[3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*. MLSys, Santa Clara, CA, USA, 351–366.
[4] Sassan Ahmadi. 2019. *5G NR: Architecture, technology, implementation, and operation of 3GPP new radio standards*. Academic Press, New York, NY, USA.
[5] AI-RAN Alliance. 2024. *Integrating AI/ML in Open-RAN: Overcoming Challenges and Seizing Opportunities*. White Paper. AI-RAN Alliance.
[6] Amazon Web Services, Inc. 2025. AWS Wavelength. https://aws.amazon.com/wavelength. Accessed: 2025-06-05.
[7] Jason et al. Ansel. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, New York, NY, USA, 929–947. https://doi.org/10.1145/3620665.3640366
[8] Apple Inc. 2024. Apple Vision Pro. https://www.apple.com/apple-vision-pro/. Accessed: 2025-06-05.
[9] Jose A Ayala-Romero, Andres Garcia-Saavedra, Xavier Costa-Perez, and George Iosifidis. 2021. EdgeBOL: Automating energy-savings for mobile edge AI. In *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21)*. ACM, New York, NY, USA, 397–410.
[10] Arjun Balasingam, Manikanta Kotaru, and Paramvir Bahl. 2024. {Application-Level} Service Assurance with 5G {RAN} Slicing. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, Santa Clara, CA, USA, 841–857.
[11] Anne Benoit, Lucas Perotin, Yves Robert, and Hongyang Sun. 2022. Online scheduling of moldable task graphs under common speedup models. In *Proceedings of the 51st International Conference on Parallel Processing (ICPP '22)*. ACM, New York, NY, USA, 1–11.
[12] Anirban Bhattacharjee, Ajay Dev Chhokra, Zhuangwei Kang, Hongyang Sun, Aniruddha Gokhale, and Gabor Karsai. 2019. Barista: Efficient and scalable serverless serving system for deep learning prediction services. In *IEEE International Conference on Cloud Engineering (IC2E '19)*. IEEE, Piscataway, NJ, USA, 23–33.
[13] Kyungmin Bin, Jongseok Park, Chanjeong Park, Seyeon Kim, and Kyunghan Lee. 2024. CoActo: CoActive Neural Network Inference Offloading with Fine-grained and Concurrent Execution. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MobiSys '24)*. ACM, New York, NY, USA, 412–424.
[14] Stephen P Boyd and Lieven Vandenberghe. 2004. *Convex optimization*. Cambridge university press, Cambridge, UK.
[15] Chieh-Chun Chen, Chia-Yu Chang, and Navid Nikaein. 2025. IUP: Integrated and Programmable User Plane for Next-Generation Mobile Networks. *IEEE Network* 39 (2025), 91–98.
[16] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. 2017. Rethinking atrous convolution for semantic image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '17)*. IEEE Computer Society, Los Alamitos, CA, USA, 14 pages.
[17] Yongzhou Chen, Ruihao Yao, Haitham Hassanieh, and Radhika Mittal. 2023. {Channel-Aware} 5g {RAN} slicing with customizable schedulers. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association, Santa Clara, CA, USA, 1767–1782.
[18] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (ATC '22)*. USENIX Association, Santa Clara, CA, USA, 199–216.
[19] NVIDIA Corporation. 2024. *Multi-Process Service (Release r550)*. NVIDIA Corporation. https://docs.nvidia.com/deploy/mps/index.html
[20] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*. USENIX Association, Santa Clara, CA, USA, 613–627.

[21] Chamitha De Alwis, Pawani Porambage, Kapal Dev, Thippa Reddy Gadekallu, and Madhusanka Liyanage. 2023. A Survey on Network Slicing Security: Attacks, Challenges, Solutions and Research Directions. In *IEEE Communications Surveys & Tutorials*. IEEE, Piscataway, NJ, USA, 534–570.

[22] Radosvet Desislavov, Fernando Martínez-Plumed, and José Hernández-Orallo. 2023. Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems* 38 (2023), 100857.

[23] Salvatore D'Oro, Leonardo Bonati, Michele Polese, and Tommaso Melodia. 2022. OrchestRAN: Network automation through orchestrated intelligence in the open RAN. In *IEEE Conference on Computer Communications (INFOCOM '22)*. IEEE, Piscataway, NJ, USA, 270–279.

[24] ETSI GS MEC 003. 2022. Multi-access Edge Computing (MEC); Framework and Reference Architecture (v3.1.1).

[25] Chengquan Feng, Li Lyna Zhang, Yuanchi Liu, Jiahang Xu, Chengruidong Zhang, Zhiyuan Wang, Ting Cao, Mao Yang, and Haisheng Tan. 2024. {LitePred}: Transferable and Scalable Latency Prediction for {Hardware-Aware} Neural Architecture Search. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, Santa Clara, CA, USA, 1463–1477.

[26] Claudio Fiandrino, Eloy Pérez Gómez, Pablo Fernández Pérez, Hossein Mohammadalizadeh, Marco Fiore, and Joerg Widmer. 2024. AIChronoLens: advancing explainability for time series AI forecasting in mobile networks. In *2024 IEEE Conference on Computer Communications (INFOCOM '24)*. IEEE, Piscataway, NJ, USA, 1521–1530.

[27] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, USA, 135–153.

[28] Apostolos Galanopoulos, Jose A Ayala-Romero, Douglas J Leith, and George Iosifidis. 2021. AutoML for video analytics with edge computing. In *IEEE Conference on Computer Communications (INFOCOM '21)*. IEEE, Piscataway, NJ, USA, 1–10.

[29] Arpan Gujarati, Sameh Elnikety, Yuxiong He, Kathryn S McKinley, and Björn B Brandenburg. 2017. Swayam: distributed autoscaling to meet slas of machine learning inference services with resource efficiency. In *Proceedings of the 18th ACM/IFIP/USENIX middleware conference (Middleware '17)*. ACM, New York, NY, USA, 109–120.

[30] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association, Santa Clara, CA, USA, 443–462.

[31] Ahmad Hassan, Shivang Aggarwal, Mohamed Ibrahim, Puneet Sharma, and Feng Qian. 2024. Wixor: Dynamic TDD Policy Adaptation for 5G/xG Networks. *Proceedings of the ACM on Networking* 2, CoNEXT4 (2024), 1–24.

[32] Hexa-X. 2022. Targets and Requirements for 6G – Initial E2E Architecture (Deliverable D1.3).

[33] Yitao Hu, Rajrup Ghosh, and Ramesh Govindan. 2021. Scrooge: A cost-effective deep learning inference system. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. ACM, New York, NY, USA, 624–638.

[34] Fatih Ilhan, Ka-Ho Chow, Sihao Hu, Tiansheng Huang, Selim Tekin, Wenqi Wei, Yanzhao Wu, Myungjin Lee, Ramana Kompella, Hugo Latapie, et al. 2024. Adaptive deep neural network inference optimization with eenet. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV '24)*. IEEE Computer Society, Los Alamitos, CA, USA, 1373–1382.

[35] Saki Imai, Rina Nakazawa, Marcelo Amaral, Sunyanan Choochotkaew, and Tatsuhiro Chiba. 2024. Predicting LLM Inference Latency: A Roofline-Driven ML Method. In *Annual Conference on Neural Information Processing Systems (NIPS '24)*. NeurIPS Foundation, Vancouver, Canada, 10 pages.

[36] Vivek Jain, Hao-Tse Chu, Shixiong Qi, Chia-An Lee, Hung-Cheng Chang, Cheng-Ying Hsieh, KK Ramakrishnan, and Jyh-Cheng Chen. 2022. L25gc: a low latency 5g core network based on high-performance nfv platforms. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '22)*. ACM, New York, NY, USA, 143–157.

[37] Rostand A K. Fezeu, Claudio Fiandrino, Eman Ramadan, Jason Carpenter, Lilian Coelho De Freitas, Faaiq Bilal, Wei Ye, Joerg Widmer, Feng Qian, and Zhi-Li Zhang. 2024. Unveiling the 5G Mid-Band Landscape: From Network Deployment to Performance and Application QoE. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '24)*. ACM, New York, NY, USA, 358–372.

[38] Taeho Kang and Youngki Lee. 2024. Attention-propagation network for egocentric heatmap to 3d pose lifting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '22)*. IEEE Computer Society, Los Alamitos, CA, USA, 842–851.

[39] Salil S Kanhere and Harish Sethu. 2002. On the latency bound of deficit round robin. In *Proceedings of the 11th International Conference on Computer Communications and Networks (ICCCN '02)*. IEEE, Piscataway, NJ, USA, 548–553.

[40] Jaehong Kim, Yunheon Lee, Hwijoon Lim, Youngmok Jung, Song Min Kim, and Dongsu Han. 2022. Outran: co-optimizing for flow completion time in radio access network. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '22)*. ACM, New York, NY, USA, 369–385.

[41] Seyeon Kim, Kyungmin Bin, Donggyu Yang, Sangtae Ha, Song Chong, and Kyunghan Lee. 2023. ENTRO: Tackling the Encoding and Networking Trade-off in Offloaded Video Analytics. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*. ACM, New York, NY, USA, 9115–9123.

[42] Woo-Hyun Ko, Ushasi Ghosh, Ujwal Dinesha, Raini Wu, Srinivas Shakkottai, and Dinesh Bharadia. 2024. {EdgeRIC}: Empowering real-time intelligent optimization and control in {NextG} cellular networks. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, Santa Clara, CA, USA, 1315–1330.

[43] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. 2015. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, New York, NY, USA, 1–6.

[44] Jinsung Lee, Sungyong Lee, Jongyun Lee, Sandesh Dhawaskar Sathyanarayana, Hyoyoung Lim, Jihoon Lee, Xiaoqing Zhu, Sangeeta Ramakrishnan, Dirk Grunwald, Kyunghan Lee, et al. 2020. PERCEIVE: Deep learning-based cellular uplink prediction using real-time scheduling patterns. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services (MobiSys '20)*. ACM, New York, NY, USA, 377–390.

[45] Baolin Li, Rohan Basu Roy, Tirthak Patel, Vijay Gadepally, Karen Gettings, and Devesh Tiwari. 2021. Ribbon: cost-effective and qos-aware deep learning model inference using a diverse pool of cloud computing instances. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*. ACM, New York, NY, USA, 1–13.

[46] Xiangjie Li, Chenfei Lou, Yuchi Chen, Zhengping Zhu, Yingtao Shen, Yehan Ma, and An Zou. 2023. Predictive exit: Prediction of fine-grained early exits for computation-and energy-efficient inference. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI '23)*, Vol. 37. AAAI Press, Washington, DC, USA, 8657–8665.

[47] Qiang Liu, Nakjung Choi, and Tao Han. 2021. OnSlicing: Online end-to-end network slicing with reinforcement learning. In *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21)*. ACM, New York, NY, USA, 141–153.

[48] Qiang Liu, Nakjung Choi, and Tao Han. 2022. Atlas: automate online service configuration in network slicing. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '21)*. ACM, New York, NY, USA, 140–155.

[49] L. Lovén, M. Bordallo López, R. Morabito, J. Sauvola, and S. Tarkoma. 2025. *Large Language Models in the 6G-Enabled Computing Continuum: a White Paper*. White Paper No. 14. University of Oulu. https://urn.fi/URN:NBN:fi:oulu-202501211268

[50] Daniel Mendoza, Francisco Romero, and Caroline Trippel. 2024. Model selection for latency-critical inference serving. In *Proceedings of the Nineteenth European Conference on Computer Systems (Eurosys '24)*. ACM, New York, NY, USA, 1016–1038.

[51] Meta Platforms, Inc. 2023. Meta Quest 3. https://www.meta.com/quest/quest-3/. Accessed: 2025-06-05.

[52] Meta Platforms, Inc. 2025. Orion: The Future of AR Glasses Technology. https://about.meta.com/realitylabs/orion/. Accessed: 2025-06-05.

[53] Microsoft Corporation. 2025. Microsoft for telecommunications. https://www.microsoft.com/industry/telecommunications. Accessed: 2025-06-05.

[54] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. 2021. Nerf: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* 65, 1 (2021), 99–106.

[55] Kelvin KW Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, New York, NY, USA, 595–610.

[56] Vinod Nigade, Pablo Bauszat, Henri Bal, and Lin Wang. 2022. Jellyfish: Timely inference serving for dynamic edge networks. In *2022 IEEE Real-Time Systems Symposium (RTSS '22)*. IEEE, Piscataway, NJ, USA, 277–290.

[57] Navid Nikaein, Mahesh K Marina, Saravana Manickam, Alex Dawson, Raymond Knopp, and Christian Bonnet. 2014. OpenAirInterface: A flexible platform for 5G research. *ACM SIGCOMM Computer Communication Review* 44, 5 (2014), 33–38.

[58] O-RAN Alliance. 2020. *O-RAN Use Cases and Deployment Scenarios: Towards Open and Smart RAN*. White Paper. O-RAN Alliance.

[59] Open-Cells. 2025. SIM Cards. https://open-cells.com/index.php/sim-cards/. Accessed: 2025-06-05.

[60] Open5GS Project. 2025. Open5GS: Open Source Implementation of 5G Core and EPC (Release-17). https://open5gs.org/. Accessed: 2025-06-05.

[61] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2024. Power-aware deep learning model serving with {μ-Serve}. In *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, Santa Clara, CA, USA, 75–93.

[62] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[63] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. 2021. {INFaaS}: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC '21)*. USENIX Association, Santa Clara, CA, USA, 397–411.

[64] Vincenzo Sciancalepore, Marco Di Renzo, and Xavier Costa-Perez. 2019. STORNS: Stochastic radio access network slicing. In *2019 IEEE International Conference on Communications (ICC '19)*. IEEE, Piscataway, NJ, USA, 1–7.

[65] Madhavapeddi Shreedhar and George Varghese. 1995. Efficient fair queueing using deficit round robin. In *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication (SIGCOMM '95)*. ACM, New York, NY, USA, 231–242.

[66] Sudipta Saha Shubha and Haiying Shen. 2023. AdaInf: Data Drift Adaptive Scheduling for Accurate and SLO-guaranteed Multiple-Model Inference Serving at Edge Servers. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '23)*. ACM, New York, NY, USA, 473–485.

[67] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. 2024. {USHER}: Holistic Interference Avoidance for Resource Optimized {ML} Inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, USA, 947–964.

[68] Sysmocom. 2025. sysmoISIM-SJA5. https://www.sysmocom.de/products/sim/. Accessed: 2025-06-05.

[69] Xin Tan, Jiamin Li, Yitao Yang, Jingzong Li, and Hong Xu. 2024. Arlo: Serving Transformer-based Language Models with Dynamic Input Lengths. In *Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24)*. ACM, New York, NY, USA, 367–376.

[70] Zhaowei Tan, Jinghao Zhao, Yuanjie Li, Yifei Xu, and Songwu Lu. 2021. {Device-Based}{LTE} latency reduction at the application layer. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI '21)*. USENIX Association, Santa Clara, CA, USA, 471–486.

[71] Jörg Tiedemann, Mikko Aulamo, Daria Bakshandaeva, Michele Boggia, Stig-Arne Grönroos, Tommi Nieminen, Alessandro Raganato, Yves Scherrer, Raúl Vázquez, and Sami Virpioja. 2024. Democratizing neural machine translation with OPUS-MT. *Language Resources and Evaluation* 58, 2 (2024), 713–755.

[72] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[73] Ethan Waisberg, Joshua Ong, Mouayad Masalkhi, Nasif Zaman, Prithul Sarker, Andrew G Lee, and Alireza Tavakkoli. 2024. Meta smart glasses—large language models and the future for assistive glasses for individuals with vision impairments. *Eye* 38, 6 (2024), 1036–1038.

[74] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, and Chen Change Loy. 2018. Esrgan: Enhanced super-resolution generative adversarial networks. In *Proceedings of the European conference on computer vision (ECCV) workshops*. Springer-Verlag, Berlin, Heidelberg, 16 pages.

[75] Yaxiong Xie, Fan Yi, and Kyle Jamieson. 2020. PBE-CC: Congestion control via endpoint-centric, physical-layer bandwidth measurements. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '20)*. ACM, New York, NY, USA, 451–464.

[76] Dongzhu Xu, Rui Lin, Huanhuan Zhang, Anfu Zhou, and Huadong Ma. 2025. Bridging Cross-Layer Interactions Between 5G RAN and MEC for Latency-Critical Video Analytics. *IEEE Transactions on Networking* Early Access (2025), 16 pages.

[77] Dongzhu Xu, Anfu Zhou, Guixian Wang, Huanhuan Zhang, Xiangyu Li, Jialiang Pei, and Huadong Ma. 2022. Tutti: coupling 5g ran and mobile edge computing for latency-critical video analytics. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking (MobiCom '22)*. ACM, New York, NY, USA, 729–742.

[78] Mengwei Xu, Zhe Fu, Xiao Ma, Li Zhang, Yanan Li, Feng Qian, Shangguang Wang, Ke Li, Jingyu Yang, and Xuanzhe Liu. 2021. From cloud to edge: a first look at public edge platforms. In *Proceedings of the 21st ACM internet measurement conference (IMC '21)*. ACM, New York, NY, USA, 37–53.

[79] Kichang Yang, Minkyung Jeong, Juheon Yi, Jingyu Lee, KyoungSoo Park, and Youngki Lee. 2024. Logan: Loss-tolerant Live Video Analytics System. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom '2024)*. ACM, New York, NY, USA, 1314–1329.

[80] Wei Ye, Xinyue Hu, Steven Sleder, Anlan Zhang, Udhaya Kumar Dayalan, Ahmad Hassan, Rostand AK Fezeu, Akshay Jajoo, Myungjin Lee, Eman Ramadan, et al. 2024. Dissecting carrier aggregation in 5G networks: Measurement, QoE implications and prediction. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '24)*. ACM, New York, NY, USA, 340–357.

[81] Juheon Yi, Goodsol Lee, Minkyung Jeong, Seokgyeong Shin, Daehyeok Kim, and Youngki Lee. 2025. Towards End-to-End Latency Guarantee in MEC Live Video Analytics with App-RAN Mutual Awareness. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '25)*. ACM, New York, NY, USA, 1–15.

[82] Juheon Yi and Youngki Lee. 2020. Heimdall: mobile GPU coordination platform for augmented reality applications. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom '20)*. ACM, New York, NY, USA, 1–14.

[83] Lanfranco Zanzi, Vincenzo Sciancalepore, Andres Garcia-Saavedra, Hans Dieter Schotten, and Xavier Costa-Pérez. 2020. LACO: A latency-driven network slicing orchestration in beyond-5G networks. *IEEE Transactions on Wireless Communications* 20, 1 (2020), 667–682.

[84] Ben Zhang, Xin Jin, Sylvia Ratnasamy, John Wawrzynek, and Edward A Lee. 2018. Awstream: Adaptive wide-area streaming analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. ACM, New York, NY, USA, 236–252.

[85] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. 2019. {MArk}: Exploiting cloud services for {Cost-Effective},{SLO-Aware} machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association, Santa Clara, CA, USA, 1049–1062.

[86] Li Lyna Zhang, Shihao Han, Jianyu Wei, Ningxin Zheng, Ting Cao, Yuqing Yang, and Yunxin Liu. 2021. Nn-meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21)*. ACM, New York, NY, USA, 81–93.

[87] Ziyang Zhang, Yang Zhao, Ming-Ching Chang, Changyao Lin, and Jie Liu. 2025. E4: Energy-Efficient DNN Inference for Edge Video Analytics Via Early Exiting and DVFS. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI '25)*, Vol. 39. AAAI Press, Washington, DC, USA, 1165–1173.

Received June 2025; accepted September 2025